

Premier League ETL pipeline project

<https://claude.ai/chat/ac9d5ed0-ddf7-45ef-8032-f505add45a34>

lets move on to the streamlit dashboard desing

Before layout, the thing that earns the 20%: pick a stakeholder and a...

what does it mean "pick a stakeholder"? should i ask myself "for who the...

Yes, exactly that — but with a second half that does the real work: **and...

i want the stakeholder to be the club analyst - the dashboard shold help...

That's a real product. "Response to the previous result" is a specific,...

i want the dashboard to allow me to pick any team and show me a...

I built the team grain and tested every candidate. Here's what has data...

we will need to create the leage table - if the team won it earns 3 points ...

Both work. I built them on your real data — and the output immediately...

consider the ranking of the team only by points already earnd - that is th...

You're right, and I should have been clearer about the distinction. Sort b...

lets try the think about the dashboard desing and how it will look like

A concrete mockup of the team page, using Brighton's real numbers fro...

the data shold update as i switch teams

Every panel is driven by your actual CSV — all 20 teams, computed from...

next tho this data i want to see the league current standing where the...

The table rows are also clickable, which turns the standings into a secon...

lets try to avoid duplicate info - if i see the table on the right with the go...

Right — and applying it properly means deciding what each region is *fo...

i want the dashboard to be a much "graphic" as possible and less "texty"

Every panel is now a shape rather than a sentence. Click through...

generate the app.py code

Every function verified against your real data. Now the app.



lets move on to the streamlit dashboard desing

6.9.2026, 1:18:57



Designing a product dashboard with clear direction. ▾

🕒 Designing a product dashboard with clear direction.

🕒 Done

Before layout, the thing that earns the 20%: pick a stakeholder and a question. A dashboard that shows results and a table is a reproduction of the Premier League website. The rubric asks what *product* you built.

The framing I'd choose

Early-season signal versus noise. You have three matchweeks. Any dashboard built on 28 matches is going to show wild-looking numbers, and the honest product is one that says so — that helps someone tell which early-season stories are real and which are small samples.

That framing turns your biggest weakness into the point of the product. A reviewer running this in September sees a dashboard that knows it's September.

The alternative — a title-and-relegation race tracker with points-per-game projections — is also good, and becomes the better choice later in the season. Pick one and say why in the README.

Structure

Four pages in a sidebar, in order of how a stakeholder would read them.

Overview. Four metric cards: matches played, goals per match, home win rate, BTTS rate — each with the sample size underneath. Then the league table, sortable, with points, GD, and form over the last 3. This is the page that must look finished; it's the first thing the reviewer sees.

Team explorer. Select a team. Their results, goals for and against by matchweek, home versus away split, and their position after each week. One team at a time keeps it legible.

Trends. Position-over-time as a line chart, one line per team, with a multiselect to cut it down — twenty lines is unreadable, so default to the top 5 plus bottom 3. Goals per matchweek. Home advantage over time.

Data quality. Your checks as a table with pass/warn status, plus a freshness stamp and the count of scheduled-but-unplayed fixtures. Most candidates hide this or skip it; putting it in the UI is what turns 15% of the grade into something visible.

Two design decisions worth making now

Cache the data load. `@st.cache_data` on a `load_matches()` function that reads the CSVs. Without it every widget interaction re-reads from disk, and the app feels sluggish in exactly the moment the reviewer is clicking around.

Never call the API from `app.py`. The assignment's run flow is `python etl.py` then `streamlit run app.py`. If the app can't find `data/processed/matches.csv`, show a clear message — "Run `python etl.py` first" — rather than crashing. That single guard is worth more than a chart.

What to build first

Overview page only, wired to the real CSV, sample sizes on every rate. Get that looking finished before adding pages. A polished single page beats four half-built ones, and the assignment says so explicitly.

You'll want the team-grain table in place before Trends — position-over-time needs it.

6.9.2026, 1:19:19



what does it mean "pick a stakeholder"? should i ask myself "for who the dashboard is desingd for?"

6.9.2026, 1:20:26



Yes, exactly that — but with a second half that does the real work: **and what decision are they making?**

"Who is it for" alone gets you to "football fans," which constrains nothing. Every chart is arguably interesting to a fan. The decision is what lets you cut things.

The difference in practice

Same data, three stakeholders:

Stakeholder	Their question	What the dashboard leads with
Sports journalist	What's the story this week?	Biggest surprises, form swings, records in reach
Betting analyst	Are the early numbers predictive?	Home advantage, BTTS rate, sample sizes and confidence
Club analyst	How are we actually doing?	One team's performance vs expectation, home/away split

A journalist doesn't need a data quality tab. A betting analyst needs error bars on everything. A club analyst wants one team foregrounded, not twenty. Three different dashboards from your 28 rows.

Why the assignment cares

The brief says the reviewer must "understand the data and insights," and the criterion is "quality of the use case, metrics, insights, UX." That phrasing is asking whether you made choices. Without a stakeholder, the default outcome is a dashboard that shows everything computable — which reads as no editorial judgment, because there wasn't any.

The stakeholder is the thing you point at when justifying a decision. "I put the league table above the trends because the journalist checks standings first" is an answer. "I put it there because it seemed natural" isn't.

A concrete test

Once you've picked, take any element on the page and ask: **what would this person do differently after seeing it?** If the answer is "nothing, but it's interesting," cut it or demote it.

That test would kill a chart of goals-per-matchweek for a club analyst and keep it for a betting analyst. Same chart, different verdict — which is the point.

My suggestion for yours

Given three matchweeks of data, I'd write something like:

This dashboard is for a football analyst deciding which early-season narratives are real. In September the sample is too small for most claims, so every rate is shown with its sample size, and the app makes it explicit when a difference is within noise.

That's one sentence in the README, and it justifies your metric choices, your error bars, your data quality tab, and what you left out. It also means the dashboard has a defensible answer to "why so few charts" — because you chose, rather than ran out of time.

Pick it before you write `app.py`. It's cheaper to design toward a decision than to retrofit a purpose onto charts you already built.

6.9.2026, 1:20:40





i want the stakeholder to be the club analyst - the dashboard should help him analyse the response of the team from week to week - how well they can keep form? how often they bounce back from defeat?

6.9.2026, 1:23:00



That's a real product. "Response to the previous result" is a specific, answerable question, and it's not what the Premier League website shows you — which is exactly what you want.

It also has a clean implication: **your grain must be one row per team per match, ordered.** Every metric you just described is a function of the *sequence* of a team's results, not of individual matches. That table is the product now, not a convenience.

The metrics that follow

Bounce-back rate. Of all matches following a defeat, what share were won? Drawn? Lost again? Same for the response to a win — do they build on it or drop off? That's a 3×3 transition matrix: previous result on one axis, next result on the other. It's the single most on-brief visual you can build, and it answers the question directly.

Streaks. Current unbeaten run, longest win streak, longest winless run. Trivial to compute once ordered, and it's how analysts actually talk.

Volatility. Points per game is the average; what a club analyst wants alongside it is the swing. A team taking 1-1-1 across three weeks and one taking 3-0-0 have identical PPG and completely different profiles. Standard deviation of points per match, or just the sequence rendered as a W-D-L strip, communicates that instantly.

Response by margin. Does a heavy defeat produce a different response than a narrow one? You have **margin** for this.

Form versus season average. Rolling 3-match points against the season rate — is the team trending up or down relative to itself.

The one column that makes all of it possible

On the team-grain table, sorted by team and matchweek:

```
df["prev_result"] = df.groupby("team")["result"].shift(1)
df["prev_margin"] = df.groupby("team")["margin"].shift(1)
```

`shift(1)` within a team group gives each match the previous match's outcome. Every metric above is a groupby on that column. Add `points`, `points_cumulative`, and a `rolling(3).sum()` of points and you have the whole analytical layer in about six lines. Note `prev_result` is null for each team's first match — that's correct, not a defect, and your quality check should assert exactly 20 nulls in that column.

Page structure

Since the stakeholder analyses *one* club, lead with a team selector at the top and let it drive everything below. Not a sidebar filter tucked away — the primary control.

Then: the W-D-L sequence strip, current streak, points per game with the season trend, the bounce-back transition matrix, and home versus away split. A league-wide comparison view second, so the analyst can see whether their club's bounce-back rate is unusual or just average.

The honesty problem, which is also your best feature

Three matchweeks means each team has played three matches, so **each team has at most two "response" observations**. A bounce-back rate computed from one prior defeat is meaningless as a rate.

Don't hide that. Two moves:

- Pool across all 20 teams for the league-wide baseline — that gives you 40 team-matches with a predecessor, which is thin but not absurd, and it's the reference the club's own number should be read against.
- Show counts, never bare percentages, at team level. "1 of 1 defeats followed by a win" is honest; "100% bounce-back rate" is not.

Build the metrics now so they become meaningful as the season runs, and put a line in the README saying the design anticipates a full season while the current sample supports only league-level aggregates. That's the kind of judgment the analytics criterion is looking for, and it turns your thinnest constraint into evidence you understood it.

6.9.2026, 1:23:21



i want the dashboard to allow me to pick any team and show me a visualization of each metric we will choose, but before we jump into desing - lets define which metrics are intresting

6.9.2026, 1:24:52



Ran command ▾

- 📄 Building the team grain and testing which metrics have data
- ⏴ Done

I built the team grain and tested every candidate. Here's what has data and what doesn't.

The numbers you actually have

56 team-matches, but only **36 have a predecessor** — that's your ceiling for anything response-related. Four teams have played only two matches, so they contribute one observation each.

The league-wide response matrix works:

After a...	Won	Drew	Lost
Win (13)	5	6	2
Draw (7)	2	5	0
Loss (16)	3	5	8

Half the teams losing again after a loss, and only 15% of losses converting straight to a win. That's a real, presentable league baseline. At team level it's 1-2 observations each — a percentage there would be dishonest.

Tier 1 — build these, they work now

Result sequence strip. **W-D-D**, **L-L-L**, **W-W-W**. Every team has one, it's instantly readable, and it *is* the answer to "how well do they keep form" for a stakeholder who reads football. Coventry's **L-L-L** and City's **W-W-W** need no chart.

Points per game with the league median. One number, one reference line.
Meaningful at $n=3$.

Cumulative points and position by matchweek. Both computable now, and the position line is your strongest chart as the season grows.

Goals for and against by matchweek. Two series, three points each. Thin but honest, and it distinguishes Brighton (8 GF, 5 GA, volatile) from Hull (3 GF, 0 GA, controlled) — a distinction their identical-looking table rows hide.

Home vs away split. Each team has 1-2 of each. Show counts, not rates.

Current streak. Trivially computable from the sequence, and it's how analysts talk.

Tier 2 — build the machinery, show at league level only

Response matrix. Compute per team, display the league version as the headline and the team's raw counts beneath it — "after 2 defeats: 1 draw, 1 loss," never a percentage.

Response by margin. Does a heavy loss produce a different response than a narrow one? The mechanism is one groupby; you have 16 post-loss observations league-wide, which supports a rough split.

Volatility. Standard deviation of points per match. At $n=3$ it's barely defined, but it's the metric that separates 1-1-1 from 3-0-0 — worth having in place.

Tier 3 — defer

Rolling 5-match form is empty at three matchweeks. Use rolling 3 now and note it.

Opponent-adjusted anything needs a strength estimate you can't build from 28 matches. **Bounce-back rate as a per-team percentage** — the machinery yes, the number no, not yet.

The design consequence

Your stakeholder wants one team, but almost every response metric needs the league as its reference. So the layout should be **team foreground, league background**: their sequence, their PPG, their response counts — each shown against the league distribution. That's what makes it analysis rather than a scorecard, and it's also what makes small samples defensible, because the league pool is 36 observations rather than 2.

One thing to decide: four teams have played twice, sixteen three times. Comparing raw points across them is misleading — City's 9 and Arsenal's 6 aren't on the same scale. Either normalize to per-game everywhere, or show **P** next to every total. Per-game is cleaner, and it's what a club analyst would expect.

6.9.2026, 1:25:23



we will need to create the league table - if the team won it earns 3 points , lose its 0 points and draw is 1 point for each team

also we can maybe create a "home table" which show which teams are dominant at home - also valid for the away -

6.9.2026, 1:29:53



Ran command ▾

- 📄 Building the league table and the home/away splits on real data
- ✓ Done

Both work. I built them on your real data — and the output immediately shows the two design problems you'd hit.

The league table

Standard columns: **P W D L GF GA GD Pts**. The points arithmetic checks out — 75 total, matching 3×28 minus 3 draws, which is a validation worth keeping as a quality check since it catches any outcome misclassification instantly.

Sort by **Pts**, then **GD**, then **GF**. That's the real Premier League tiebreak order (head-to-head only comes in after those, and it's not worth implementing).

The problem: four teams have played twice, sixteen three times. Arsenal on 6 points from 2 sits below Hull on 7 from 3, but Arsenal's PPG is 3.00 and Hull's is 2.33. The raw table is actively misleading right now.

Two fixes, and I'd do both: always show **P**, and offer a PPG sort toggle. For a club analyst, points-per-game is the honest comparison in September and it costs one radio button.

Home and away tables

Same shape, filtered by venue. And this is where your product idea pays off — the two tables have almost no overlap in their top six. Brighton and Brentford are strong at home and absent from the away list; Liverpool and Leeds are the reverse.

The problem is worse here though: twelve teams have played just *one* home match. A "home table" where most rows are a single game isn't a table, it's a

scoreboard.

So present it as a **comparison, not a ranking**. For the selected team, show home PPG next to away PPG with the match counts attached, and a league-median reference line. "Home 2.0 (2 matches), away 3.0 (1 match)" is honest. A ranked home table with 1-match rows invites conclusions the data can't support.

Later in the season, when everyone has 8-10 of each, the ranked version becomes legitimate — so build the machinery now and gate the presentation on sample size. Something like: show the ranked table only when the minimum home matches across teams is at least 3, otherwise show the comparison view with a note. That's a small conditional that demonstrates statistical judgment, and it self-resolves as the season runs.

Implementation note

Both tables are the same function with a filter:

```
def build_table(team_matches, venue=None):  
    df = team_matches if venue is None else  
    team_matches[team_matches.venue == venue]  
    ...
```

One function, three calls. Write it in `src/analytics.py`, not in `app.py` — that keeps the metric logic testable and stops the Streamlit file from becoming where the business rules live. Your `/tests/` folder can then assert that a known fixture produces a known table, which is the cheapest meaningful test you can write.

6.9.2026, 1:30:18



consider the ranking of the team only by points already earned - that is the proper way in all the sporting websites and apps

6.9.2026, 1:31:41



You're right, and I should have been clearer about the distinction. Sort by total points — that's how every table works, and inventing a PPG-sorted table would look like you didn't know the convention.

So: **Pts** desc, then **GD**, then **GF**. Standard.

The thing I was reaching for is separate from sort order. Football sites can sort by points alone because in a normal season every team has played the same number of matches — the column exists but it's uniform. Your table has **P** values of 2 and 3 in the same view, because MW3 is half-played. Arsenal sit 3rd on 6 points with a game in hand on Hull above them on 7.

That's not a reason to change the ranking. It's a reason to make the games-in-hand visible, which is exactly what real sites do:

- Keep **P** as a column — it's already in the standard layout and it carries the information
- Optionally mark rows with fewer matches than the maximum, the way broadcasters footnote "game in hand"

And PPG becomes just another column a club analyst can look at, not the sort key. It's genuinely useful to them — comparing their team's rate against the league — but it sits alongside the table rather than reorganizing it.

That keeps the table conventional and still honest. The ranking is the ranking; the context is a column.

6.9.2026, 1:31:49



lets try the think about the dashboard desing and how it will look like

6.9.2026, 1:32:28



Building a mockup team page with real match statistics. ▾

🕒 Building a mockup team page with real match statistics.

✅ Done

A concrete mockup of the team page, using Brighton's real numbers from your file so you can see how thin the data actually looks in a layout.

Team **Brighton & Hove Albion ▾**

Season 2026/27 · after MW3

Position	Points	Points per game	Goal difference
9th	4	1.33	+3
of 20	3 played	league median 1.42	8 scored, 5 conceded

Form

W L D MW1 to MW3 · currently 1 unbeaten

Beat Aston Villa 4-0, lost 4-3 at Chelsea, drew 1-1 with Leeds

Home

2.00 points per game

2 played · 1W 1D 0L · 5 scored, 1 conceded

Away

0.00 points per game

1 played · 0W 0D 1L · 3 scored, 4 conceded

Response to the previous result

After a win After a defeat After a draw

1 loss 1 draw no data yet

League baseline across 36 team-matches with a predecessor: 50% of defeats are followed by another defeat, 19% by a win.

Four layout decisions worth defending, and one problem the mockup exposes.

The team selector is the page, not a sidebar filter. Your stakeholder analyses one club. Putting the control at the top, above everything it drives, makes the causality obvious. Streamlit's default instinct is `st.sidebar.selectbox` — resist it here.

Every rate carries its denominator. "1.33 points per game" alone is a number; "1.33, league median 1.42" is a judgement. "2 played" under the home split is what stops a reviewer thinking you've overclaimed. This is the pattern to apply everywhere — the comparison and the count, never the bare figure.

Form is a strip, not a chart. Three coloured pills communicate more to a football person than any visualization of three data points would. Resist charting things that fit in a sentence.

The response panel shows counts and defers to the league. "1 draw after a defeat" is honest. "100% bounce-back rate" from one observation would be embarrassing. The league baseline underneath — 36 team-matches, 50% of defeats followed by another defeat — is the real finding, and it's the reference the club's own numbers get read against.

The problem

Look at how much white space three matches produces. Half these panels are one number and a caption. That's not a design failure, it's the data — and it's why the "no data yet" on after-a-draw matters. Showing it, rather than hiding the panel, tells the reviewer you know what's missing.

Two ways to handle it, and I'd do both: keep the panels and let them fill as the season runs, and add a league-comparison view as a second page so there's somewhere with enough data to say something substantial.

What to build next

Get this one page working against your real CSV before adding anything. Then in order: the position-over-time chart (needs cumulative rank on the team grain), the league comparison page, and the data quality tab.

One implementation note: put every metric in `src/analytics.py` as functions taking a DataFrame and returning a DataFrame. `app.py` should only select, call, and render. That keeps the logic testable and stops Streamlit from becoming where your business rules live.

6.9.2026, 1:33:07



the data shold update as i switch teams

6.9.2026, 1:33:53



Ran 2 commands ▾

- 📄 Computing per-team stats for all 20 teams to embed in the mockup
- 📄 Compacting the JSON for embedding
- ✅ Done

Team **Brighton & Hove Albion** ▾ Season 2026/27 · after MW3

Position	Points	Points per game	Goal difference
9th	4	1.33	+3
of 20	3 played	league median 1.42	8 scored, 5 conceded

Form

W L D currently 1 unbeaten

W 4-0 v Aston Villa

L 3-4 at Chelsea FC

D 1-1 v Leeds United

Home	Away
2.00 points per game	0.00 points per game
2 played · 1W 1D 0L · 5 scored, 1 conceded	1 played · 0W 0D 1L · 3 scored, 4 conceded

Response to the previous result

After a win After a draw After a defeat

1 loss no data yet 1 draw

League baseline across 36 team-matches with a predecessor: 50% of defeats are followed by another defeat, 19% by a win.

Every panel is driven by your actual CSV — all 20 teams, computed from the same team-grain transformation you'll build in `analytics.py`. Click through a few and you'll see what the layout does and doesn't handle.

Worth clicking specifically:

Coventry and Fulham — `L-L-L`, zero points, "no unbeaten run". The empty states have to read as deliberate, not broken.

Arsenal and Chelsea — 2 played, so the "of 20" position is doing work the points total can't. This is where games-in-hand matters.

Manchester City — after a win: 2 wins. Their response panel has data because they've won every game. Most teams show "no data yet" in at least one row, which is honest and will fill in over the season.

What this tells you about the Streamlit build

Everything the mockup renders comes from one dict per team. In Streamlit that's one function:

```
def team_summary(team_matches, team):  
    ... # returns a dict of exactly these fields
```

`app.py` then does `d = team_summary(tm, selected)` and renders. No pandas in the view layer, and the function is directly testable against a fixture.

Streamlit reruns the whole script on every widget change, so the team selector "just works" — but only if the CSV read is wrapped in `@st.cache_data`. Without it you re-read from disk on every click.

What's missing from this page

No position-over-time chart, because you need cumulative rank per matchweek — that's the next thing to add to the team grain. It's also the panel that will look best in three months and worst today, which is an argument for putting it below the fold.

And the league baseline at the bottom is currently a hardcoded sentence. Compute it, so it updates as the season runs.

6.9.2026, 1:35:30



next tho this data i want to see the league current standing where the chosen team is highlited inside the table

6.9.2026, 1:40:54



Team

Season 2026/27 · after MW3

Position

Points

Points per game

Goal difference

—

—

—

—

of 20

—

—

—

Form

Home

Away

— points per game

— points per game

—

—

League standings

Team

P W D L GD Pts

Sorted by points, then goal difference, then goals scored. P varies because matchweek 3 is part played.

The table rows are also clickable, which turns the standings into a second selector — useful when the analyst wants to check who's immediately above or below them.

Implementing the highlight in Streamlit

`st.dataframe` supports row styling through pandas Styler:

```
def highlight_team(row, team):
    is_sel = row.name == team
    return ["background-color: rgba(55,138,221,0.15)" if is_sel else
            "" for _ in row]

st.dataframe(
    table.style.apply(highlight_team, team=selected, axis=1),
    use_container_width=True,
)
```

Two gotchas. `st.dataframe` ignores most Styler formatting but does honour `background-color`, so keep it to that. And use a translucent `rgba` rather than a solid colour — a solid fill breaks in dark mode, where the default text colour stops being readable against it.

If the styling fights you, the fallback that always works is `st.table` with pre-rendered HTML, or just adding a  marker column on the selected row. Less pretty, zero risk.

Two things the table exposes

The  column is carrying real weight. Arsenal and Chelsea sit 3rd and 4th on 6 points with two games played; Hull are 2nd on 7 from three. Anyone reading the table without noticing  draws the wrong conclusion. Consider a small note or a marker on rows with fewer matches than the league maximum — that's what broadcasters do.

Nothing indicates the European or relegation cut-offs. Real tables use a coloured left border to separate positions 1–4, 5, and 18–20. It's cosmetic, but for a club analyst it's the first thing their eye looks for. One extra CSS rule if you go the HTML route; skip it if you use `st.dataframe`.

Where this leaves the build

You now have a specified page. In `analytics.py` you need three functions:

`build_team_matches(matches)`, `build_table(team_matches, venue=None)`, and `team_summary(team_matches, team)`. Everything on screen comes from those. Write them with tests against a small fixture, then `app.py` is mostly layout.

The remaining gaps are unchanged: `quality.py`, the `extract` hardening, and position-over-time once you add cumulative rank.

6.9.2026, 1:42:10



lets try to avoid duplicate info - if i see the table on the right with the goal diff and team position i have no need to show it again in a different part of the dashboard

6.9.2026, 1:44:59



Redesigning stat cards to complement the table's data. ▾

- 🕒 Redesigning stat cards to complement the table's data.
- 🕒 Done

Right — and applying it properly means deciding what each region is *for*. The table answers "where do they stand." The panel should answer only what the table can't: how they got there and how they respond.

Team

2026/27 · after MW3

Form

Standings

Rate against the league

— points per game

—

Team P GD Pts

Points, then goal difference, then goals scored. P varies because MW3 is part played.

Home and away

Home

Away

—

—

—

—

Response to the previous result

League baseline, 36 team-matches with a predecessor: 50% of defeats followed by another defeat, 19% by a win.

What got cut and why:

Position, points, and goal difference are gone from the panel — all three are in the table, one glance away. The four metric cards are gone entirely; three of them were duplicating table columns.

W/D/L totals are gone from the table, since the form strip and the home/away records carry that information in more useful form. The table is now the canonical minimum: `# Team P GD Pts`.

Points per game survives because it's the one rate the table can't express — and it's now framed as a comparison, "+/– against the league median," rather than a bare number. That's the version an analyst can act on.

Goals scored and conceded dropped out of both. They're implicit in the results list and summarized by GD in the table. If you find you want them, put them in the home/away card where the split is actually informative, not in a headline card.

The panel now reads as four questions the table can't answer: how are they trending, what rate are they at relative to everyone, does venue matter, and how do they respond. That's the product.

Two things to watch as you build this

The de-duplication rule has a limit. Position appears nowhere in the panel now, which is right — but if you later add a page without the table, it has to come back. Tie the rule to the *view*, not to the metric.

And the results list under the form strip is the one place still repeating something: the strip says `W-L-D` and the list says the same in words with scores. I'd keep it. The strip is for scanning, the list is for reading, and they're at different levels of detail rather than being the same fact twice.

6.9.2026, 1:46:21



i want the dashboard to be a much "graphic" as possible and less "texty"

6.9.2026, 1:47:50



Designing visual charts to make the season stats vivid. ▾

- 🕒 Designing visual charts to make the season stats vivid.
- 📧 Computing per-matchweek series for the graphic version
- 👍 Done

Team ▾

2026/27 · MW3

Points accumulated

Standings

Team GD P Pts

Bar length is goal difference

Goals by matchweek

Scored above the line, conceded below

Home and away

Response to the previous result

Darker cells are more frequent. Empty means no occurrences yet.

Every panel is now a shape rather than a sentence. Click through Coventry, Man City, and Crystal Palace — the differences read instantly.

Four things that make these charts carry meaning rather than decorate:

The points curve has a reference line. The dashed line is league pace at 1.42 points per game. Without it, a rising line is just a rising line — every team's cumulative points go up. Against pace, you can see immediately whether a club is gaining or falling behind. Crystal Palace's flat-then-jump shape versus Liverpool's steady climb tells you something a number can't.

Goals diverge around a baseline. Scored above, conceded below. Aston Villa's chart is all downward bars with nothing above the line — zero goals in three games. That's a diagnosis in one glance.

The home/away bars share a scale and a median marker. Two bars on a 0–3 axis, so the comparison is spatial. Brighton's 2.00 home versus 0.00 away is a gap you see, not one you compute.

The response grid is a heatmap, not a table. Rows are the previous result, columns the next, colour intensity is frequency. Fulham's dark cell at loss-to-loss and Man City's at win-to-win are the whole story of those clubs so far.

Porting to Streamlit

Two viable routes. **Altair** is built in and declarative, which suits the line and bar charts — `st.altair_chart` with a `mark_line` plus a `mark_rule` for the pace line. The response grid is `mark_rect` with a colour encoding, which is a natural fit.

Or keep the hand-drawn SVG and render via `st.html`. More control, and it's what you see here. Given you'll want the pace line, the diverging baseline, and the median marker — all things Altair supports but with some fighting — I'd try Altair first and fall back to SVG for the grid if it resists.

Either way, the GD bars in the table are simplest as a Styler `bar` column:

```
table.style.bar(subset=["GD"], align="zero", color=["#E24B4A",  
"#1D9E75"])
```

One caution

At three matchweeks the line chart has three points. That's a shape, not a trend, and the eye reads more into a line than the data supports. Keep the pace reference — it's what stops the chart overclaiming — and consider showing points as a step chart rather than a smooth line so it reads as discrete events.

6.9.2026, 1:49:35



generate the app.py code

6.9.2026, 1:56:14



Created file, ran command ▾

- 📄 Creating analytics.py with the metric functions the dashboard needs
- 📄 Validating analytics.py against the real data
- ✓ Done

Every function verified against your real data. Now the app.

Created file, ran command, presented file ▾

- 📄 Creating app.py, the Streamlit dashboard
- 📄 Checking syntax and exercising every code path for all 20 teams
- 📄 Presenting file(s)...
- ✓ Done



app
Py



analytics
Py



6.9.2026, 1:57:37